Image based breed recognition for cattle and buffaloes of India progressive web app

Submitted in the partial fulfillment of the requirements

for the degree of B.Tech in Information Technology

by

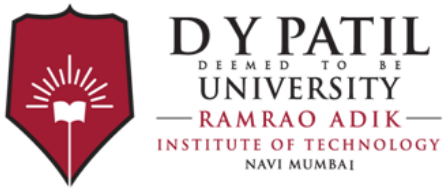
Ms. Shravani Desai (22IT1066)

Ms. Swara Gawande (22IT1149)

Mr. Kunal Dhaigude (22IT1114)

Supervisor

Dr. Prajakta Dere



Department of Computer Engineering

Ramrao Adik Institute of Technology

Sector 7, Nerul, Navi Mumbai

(Under the ambit of D. Y. Patil Deemed to be University)

November 2025

Ramrao Adik Institute of Technology

(Under the ambit of D. Y. Patil Deemed to be University)

Dr. D. Y. Patil Vidyanagar, Sector 7, Nerul, Navi Mumbai 400 706

CERTIFICATE

This is to certify that, the Minor-DS Mini Project report entitled

Image based breed recognition for cattle and buffaloes of India progressive web app

is a bonafide work done by

Ms. Shravani Desai (22IT1066)

Ms. Swara Gawande (22IT1149)

Mr. Kunal Dhaigude (22IT1114)

and is submitted in the partial fulfillment of the requirement for the degree of

B.Tech in Information Technology

to the

D. Y. Patil Deemed to be University

Supervisor	Project Co-ordinator
(Dr. Prajakta Dere)	**(Dr. Tushar H. Ghorpade)**
Head of Department	Principal
(Dr. Amarsinh V. Vidhate)	**(Dr. Mukesh D. Patil)**

Minor-DS Mini Project Report Approval

This is to certify that the Minor-DS Mini Project entitled ***A Deep Learning Approach for Indigenous Indian Cattle Breed Classification*** is a bonafide work done by ***Shravani Desai (22IT1066)***, ***Swara Gawande (22IT1149)***, ***Kunal Dhaigude (22IT1114)***, under the supervision of ***Dr. Prajakta Dere***. This Mini Project is approved in the partial fulfillment of the requirement for the degree of ***B.tech in Information Technology***

Internal Examiner :

1. .....

2. .....

External Examiners :

1. .....

2. .....

Date : .../.../.....

Place : .....

DECLARATION

I declare that this written submission represents my ideas and does not invovle plagiarism. I have adequately cited and referenced the original sources wherever others' ideas or words have been included. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action against me by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date: _____

Ms. Shravani Desai (22IT1066)

Ms. Swara Gawande (22IT1149)

Mr. Kunal Dhaigude (22IT1114)

Abstract

This project is tackling the very real problem of identifying indigenous Indian cattle breeds, which is currently accomplished through subjective, manual expertise incapable of scaling. The main aim was to develop, train, and assess an automated, deep learning classifier so that identification could be a more objective and accessible methodology. We developed a systematic analysis of a baseline Convolutional Neural Network (CNN) against three powerful transfer learning architectures - ResNet18, DenseNet121, and EfficientNet-B0. These models were trained in a Google Colab environment using the PyTorch framework on a custom image dataset of 41 cattle breeds.

This goal accomplished. The first model comparison clearly illustrated the improvement of EfficientNet-B0, reaching a validation accuracy of 57.35 percent, much higher than the baseline CNN and other pre-trained models. The model was fine-tuned using a multi-stage fine-tuning process, increasing the peak validation accuracy to 62.79 percent. The final baseline model performed at 55.90 percent accuracy on the blind test set, and a per-class analysis indicated a strong performance on the high-sample breeds, whereas class imbalance was the main hurdle for future work. The project concludes with an inference module capable of classifying new cattle images from a URL, supporting the feasibility of this approach as a foundation for a functionally exploitable product for farmers and conservationists.

Contents

List of Figures

Chapter 1

Introduction

Cross-breeding and ineffective conservation efforts have placed India's indigenous cattle — highly valued for their adaptability, disease resistance, and milk quality – at risk. A crucial issue is the timely and accurate identification of breeds, which is impeded by the visual similarity of the breeds. Because of this difficulty in identification, farmers are unable to breed for genetic purity, increase market value, or effectively manage the breed. This project proposes to develop an **automated classification system using computer vision and deep learning** to identify indigenous cattle breeds accurately with digital photos.

1.1 Overview

The main goal of this project is to develop, implement, and assess an **Indigenous Cattle Breed Classifier**. The proposed system is designed to serve as a valuable tool for a reliable and time-efficient breed identification process that uses the most advanced machine learning techniques as a methodology.

The project aims to investigate the effectiveness of deep learning algorithms, and more specifically Convolutional Neural Networks (CNNs), in identifying the subtle visual characteristics that differentiate cattle breeds. The methodology involves:

- The collection and preprocessing of a multiple-class image dataset of indigenous cattle.
- The development and training of a baseline CNN model to assess baseline performance.
- The implementation of transfer learning to fine-tune the CNN architectures ResNet18, DenseNet121, and EfficientNet-B0 to the present task.

- The assessment of the models to determine the optimal classifier in terms of performance and accuracy.

Therefore, you will end up with a model that is trained to predict a cattle's breed using a new image, laying the groundwork for an application that could ready for deployment (e.g., mobile app) and used directly in the field by stakeholders.

1.1.1 AI & ML

Using Artificial Intelligence (AI) and specifically Machine Learning (ML) will solve this problem. Traditional programming relies on rules that have been explicitly written, which would be unmanageable for a task such as breed identification (for example, "if the curvature of horn **shape is $>$ 30 degrees and size of hump is 'large', etc."). ML, or** specifically deep learning, flips this model on its head.

Instead of being explicitly programmed, a deep learning model relies on the model initially learning the identifying rules of the application domain directly from data. While there are several deep learning models (e.g. deep belief networks, recurrent neural networks, neural topic modelling) we focus on Convolutional Neural Networks (CNNs) – a class of deep learning model that processes and is tailored to visual information. CNNs learn a hierarchy of features automatically:

1. **Low-level features:** Edges and textures (e.g., hide patterns, hair texture).
2. **Mid-level features:** Combinations or arrangements low level of features (e.g., shape of the ear, curve shaped horn).
3. **High-level features:** More complex parts of an object (e.g. the identifiable distinct hump of a Sahiwal, head-shaped contour of a Gir).

This project employs **Transfer Learning** to achieve high accuracy while avoiding requiring millions of images. In essence, Transfer Learning "transfers" knowledge from a model (i.e., EfficientNet-B0) that was previously trained on a large, generic dataset (ImageNet) and fine-tunes it on our unique cattle dataset. In doing so, the model is able to use its pre-learned vision to identify general visual features in order to learn the unique task of distinguishing cattle breeds.

1.2 Motivation

The obstacles that were discussed in the introduction have substantial real-life implications, and continue to serve as the primary motivation for this work.

1. **Economic Empowerment:**The predominant motivation is the economic inequity that farmers experience. Farmers' market access for indigenous cattle depends greatly on breed uniformity. A Gir or Sahiwal cow that is certified will have a much higher price than an equivalent cow/ox that is un-verified. As a consequence, when small-scale farmers do not have an accessible and reliable means of breed verification, they often do not receive an equitable price from a market-based economy and are subsequently disadvantaged. An automated classifier would serve as a vehicle for economic empowerment by providing an objective means of breed verification.

2. **Genetic Conservation:** The second, equally important motivation is the accelerating loss of genetic diversity. Cross-breeding, often the result of misidentification, leads to the loss of ecologically adapted and cause-oriented genetic lines. Organizations would like to have good data to measure and make selections for breeding programs, keep herd books, and conserve purebred dairy populations. An automated identification tool can be used at scale for large scale data collection and monitoring and, therefore, for the preparation of effective conservation strategies.

3. **Accessibility and Objectivity:**The current "gold standard" for identifying breeds is a human expert. This dependence on specialized, experiential knowledge generates tremendous bottlenecks. The information from experts is subjective, not easily scalable, and often completely unobtainable to a regular farmer in a remote village. This research is driven by the urgent need to democratize this knowledge. A deep learning model, available in a simple application, would provide anyone with a smartphone an objective, accurate, and instantaneous classification, thereby bridging the critical knowledge gap.

1.3 Problem Statement and Objectives

1.3.1 Problem Statement

The issue is that we do not have an **accurate, scalable,** and **accessible automated means to classify indigenous Indian cattle breeds from digital images.** The manual classification process is reliant on subjective interpretation, inconsistent, error-prone, and requires specialistic expertise that is not widespread. This information gap causes financial implications for farmers, impedes effective management of livestock based on breed traits, and can also damage the credibility of national genetic conservation programs. As such, this project addresses the technical challenge of developing a rigorous, deep learning-based classifier to deliver a dependable solution to the problem.

1.3.2 Objectives

To respond to the problem statement, this project will carry out the following specific, measurable tasks:

1. **Dataset Curation:** To gather, organize, and prepare a comprehensive image dataset of indigenous cattle breeds, located at /content/drive/MyDrive/cattle images whole, and create a solid data preprocessing and augmentation pipeline.

2. **Baseline Modeling:** To utilize and train a baseline Convolutional Neural Network (CNN) from scratch to obtain a baseline performance metric, and to understand the difficulties of the dataset itself.

3. **Transfer Learning Evaluation:** To apply and systematically test several state-of-the-art pre-trained transfer learning architectures, namely **ResNet18**, **DenseNet121,** and **EfficientNet-B0**, to determine the most effective feature extractor for this task.

4. **Model Optimization:** To optimize the best model (EfficientNet-B0) with state-of-the-art fine-tuning functions, smaller learning rates, and a broader range of data augmentations to maximize classification accuracy.

5. **Inference Module Development:** To develop a practical inference module for the fully trained model, that is able to accurately predict a cattle breed from an unfamiliar image provided as either a local file or a web-based URL.

1.4 Organization of the Report

The structure of this document is comprised of five chapters that provide a description of the architecture and evaluation of the Indigenous Cattle Breed Classifier.

- **Chapter 1** is an introduction to the problem area. It outlines the real-world motivation for an automated cattle classifier, including both economic and conservationist motivations. It also defines the formal problem statement and describes the technical goals of the project.

- **Chapter 2** is a literature review of the surrounding literature related to computer vision for agriculture and deep learning. It covers livestock identification studies, deep learning models specifically Convolutional Neural Networks - CNNs, and transfer learning ideas.

- **Chapter 3** escribes the system architecture and methodology. The chapter describes the hardware and software environment (Google Colab, PyTorch), the data preparation process, data split and augmentation methods, and model architecture for the proposed models.

- **Chapter 4** presents the core implementation, experiment, and results. It describes the training and evaluation of the baseline CNN model. The results of the transfer learning comparative study of models **ResNet18, Densenet121**, and **EfficientNet-B0** are presented next. Lastly, the fine-tuning process and evaluation of the highest performance model **EfficientNet-B0** is described.

- **Chapter 5** showcases an inference module to demonstrate the operationalization of the final model. In addition, it discusses the final findings and limitations of the project, and suggests some avenues for future work, such as deployment and further improvements to the model.

Chapter 2

Literature Survey

The main goal of this chapter is to summarize the suggestive literature and current systems that led to the discovery and eventual selection of the task for this project. This summary identifies the primary non-technical strategies utilized for cattle identification, reviews existing technical solutions and processes, and pinpoints several key deficiencies and research opportunities that this study seeks to address.

2.1 Survey of Existing Systems

There is a clear progression of foundational research on which the current systems in this field are based. The survey starts with the fundamental, all-purpose instruments that enabled contemporary computer vision. This includes the foundational deep learning architectures of **ResNet** [2] and **DenseNet** [3], which addressed the problem of training very deep networks, as well as the groundbreaking **ImageNet dataset**, which supplied the massive amount of data required for training. A significant system milestone was reached with the release of EfficientNet [4], which sets a new standard for striking a balance between model accuracy and computational cost.Another important ”system” or framework for putting these models into practice is the PyTorch library [5].

The fields of agriculture and livestock soon adopted these fundamental instruments. Among the current systems are general-purpose **AI-powered livestock recognition** [6] and, more precisely, the identification of cattle breeds using standard **CNNs** [7], [8]. These ”existing systems” for cattle identification have been the focus of comparative studies [9] and have been tried with alternative image processing and **machine learning methods** [10] because the field is suf-

ficiently developed. General **animal species recognition** [11] and the modification of these models, like EfficientNet, for other intricate biological identification tasks, such as medical image analysis [12], are also included in the range of current applications.

2.2 Limitations of Existing Systems

Although the aforementioned systems are operational, the following papers' focus is determined by their limitations.

Limitation 1: Insufficient Specificity. The first limitation is the lack of specificity. A straightforward classification of a common breed [7, 8] is neither adequate nor "universal." Deep mathematical learning for "universal bovine identification" [13], which can learn to differentiate between individuals as well as breeds, is proposed as a solution to this limitation.

Limitation 2: Sub-optimal Algorithms. Algorithms that are not optimal. For some complex identification tasks, a standard CNN with only one model might not be the best algorithm. A "Hybrid Deep Learning Algorithm" for dog breed identification [14] addresses this, indicating that combining models is a useful strategy. Although the fundamental issue of animal classification [11] has been resolved, it remains a persistent problem that is still being improved [15].

Limitation 3: Breed vs. Individual Identification. Breed vs. Individual Identification is the third limitation. It is essential for management to distinguish between identifying a breed and identifying a particular individual animal. The "Non-Invasive Muzzle Matching" technique [16], which goes from breed-level to individual-level recognition, directly addresses this limitation.

Limitation 4: Computational Inefficiency. Computational inefficiency is the fourth limitation. For on-farm or embedded deployment, many deep learning models [2, 3] are computationally demanding and ineffective. An "Efficient Framework" that makes use of a "Multi-Part CNN" [17] to increase speed and accuracy is proposed in order to overcome this limitation.

Limitation 5: Lack of Real-World Robustness. The fifth limitation is the absence of real-world robustness. When used in uncontrolled, real-world settings, models that were trained

on pristine, carefully selected datasets [1] perform poorly. ”Efficient Method for Categorize Animals in the Wild” [18] focuses on this. Applying deep learning to livestock farming is a broad and difficult field, as further supported by a systematic literature review [19].

Limitation 6: Lack of Model Focus. The sixth limitation is the absence of model focus. Conventional CNNs may be ”dumb” and fail to highlight an image’s key elements. In order to help models focus, ”efficient neuron attention” [20] is introduced. In [21], this idea is developed using a ”Dual Attention Network” to examine an animal’s behavior (such as feeding) in addition to its characteristics.

Limitation 7: Poor Data Quality. The seventh limitation is low-quality data. Using outdated, ineffective models (such as VGG) continues [22]. More significantly, real-world data is frequently of low quality. This restriction is the specific objective of ”Using EfficientNet for... Images with Low Resolution” [23]. The pursuit of Other ”Hybrid Deep Learning” techniques continue to produce better models [24].

Limitation 8: Reliance on Visual Data. Limitation 8: Dependency on Visual Information. The field as a whole, from [1] to [24], is essentially constrained by its dependence on visual information (pictures). In order to address this, [25] suggests a deep learning approach for breed identification utilizing genomic prediction, which is an entirely new and possibly more trustworthy modality.

2.3 Conclusion: Observations and Identified Limitations

2.3.1 Observations

To address the simplest issue, cattle breed identification [6–10], the field first builds its toolkit by utilizing foundational models [1–5]. The limitations of these early systems are then promptly identified by the research.

2.3.2 Identified Limitations

The remaining bibliography directly attacks these restrictions, with each paper or collection of papers focusing on a particular gap:

1. **The Specificity Limitation:** Research shifts from breed-specific identification [7, 8] to individual-specific recognition [16] and universal recognition [13].

2. **The Algorithm Limitation:**standard CNNs [11, 15] give way to intelligent Attention-based networks [20, 21] and more sophisticated Hybrid models [14, 24].

3. **The Efficiency Limitation:** Replacing outdated models [22] with "Efficient" frameworks [4, 17] and techniques [18] is a primary focus.

4. **The Robustness Limitation:** The need to transition from "lab" to "field," which is accomplished by addressing "in the wild" [18] and "low-resolution" [23] scenarios, is a crucial gap in robustness.

5. **The Modality Limitation:** Lastly, the last paper [25] points to a new frontier in genomics, highlighting the survey's overall reliance on visual data as a limitation.

Chapter 3

Methodology and System Design

This chapter discusses the framework, tools, and overall procedure in the technical development of the Indigenous Cattle Breed Classifier. We will sketch the system architecture in broad terms, speak about the environment the system has been developed in, and provide information about the software libraries utilized as we move from data inputs to a trained and evaluated model.

3.1 Proposed System Architecture

To contribute to the issue of providing an affordable, accurate and scalable tool for cattle breed identification, we propose an automated classification system based on deep learning. The system is designed as a complete pipeline, from data intake to predictions made in real time, constructed utilizing state-of-the-art, computer vision techniques.

The envisioned system encompasses the following aspects:

- **Cloud-Based Development:** The entire architecture is created within Google Colab in order to leverage its free tier GPU acceleration which is essential for training deep neural networks.

- **Persistent Storage:** Within the Colab environment, Google Drive is leveraged for persistent storage of the dataset (i.e., /content/drive/MyDrive/cattle images whole), and just as, if not more importantly, for model checkpoint persistency
  (i.e. /content/drive/MyDrive/efficientnet-b0 best.pth). This ensures that we do not lose any progress, if there are disconnections during runtime.

- **Transfer Learning Core:**The brain of the system relies on a deep Convolutional Neural

Network (CNN) rather than developing a network from scratch, which would necessitate an enormous dataset, we take advantage of transfer learning where we leverage models trained on the ImageNet dataset [1] to conduct our task.

- **Systematic Model Comparison:** The architecture is also designed to be modular, so that we can evaluate multiple pre-trained models. In this project we measure a CNN baseline model against ResNet18 [2], DenseNet121 [3], and EfficientNet-B0 [4] to empirically identify the highest performing model for our best-performing model.

- **End-to-End Pipeline:** The architecture supports the entire machine learning lifecycle:

  1. **Data Preparation:** Loads, preprocesses, and augments image data.
  2. **Model Training:**Runs training and validation loop .
  3. **Model Evaluation:** Evaluates models using validation accuracy and loss.
  4. **Inference:** Evaluates models using validation accuracy and loss. URLs.

In Figure 3.1 we have depicted the top-level workflow of the proposed system, which outlines the pathway from an input image to a final breed prediction.

Transfer learning is helpful, as the number of images per breed is fairly limited. The beauty of the pre-trained model is that its parameter space is capturing a highly valuable set of generalized visual features (e.g., edges, textures, and shapes) as it has been trained on millions of images. The entire system is really just fine-tuning the generalized features of the model and re-training the final classification layer because an already pre-trained model will be considerably more data efficient and possess more accuracy than the simple CNN previously displayed in Chapter 4. Notably, for this project, EfficientNet-B0 [4] was elected as our finetuning model candidate since it consistently produced high accuracy and computational efficiency relative to the state-of-the-art pre-trained model architecture.

3.2 Hardware and Software Requirements

The project was implemented and developed entirely in Google Colab, which means that the hardware and software requirements are dependant on what Google provides through that service. In order to obtain the same results, you must have a similar hardware and software configuration, as described below:

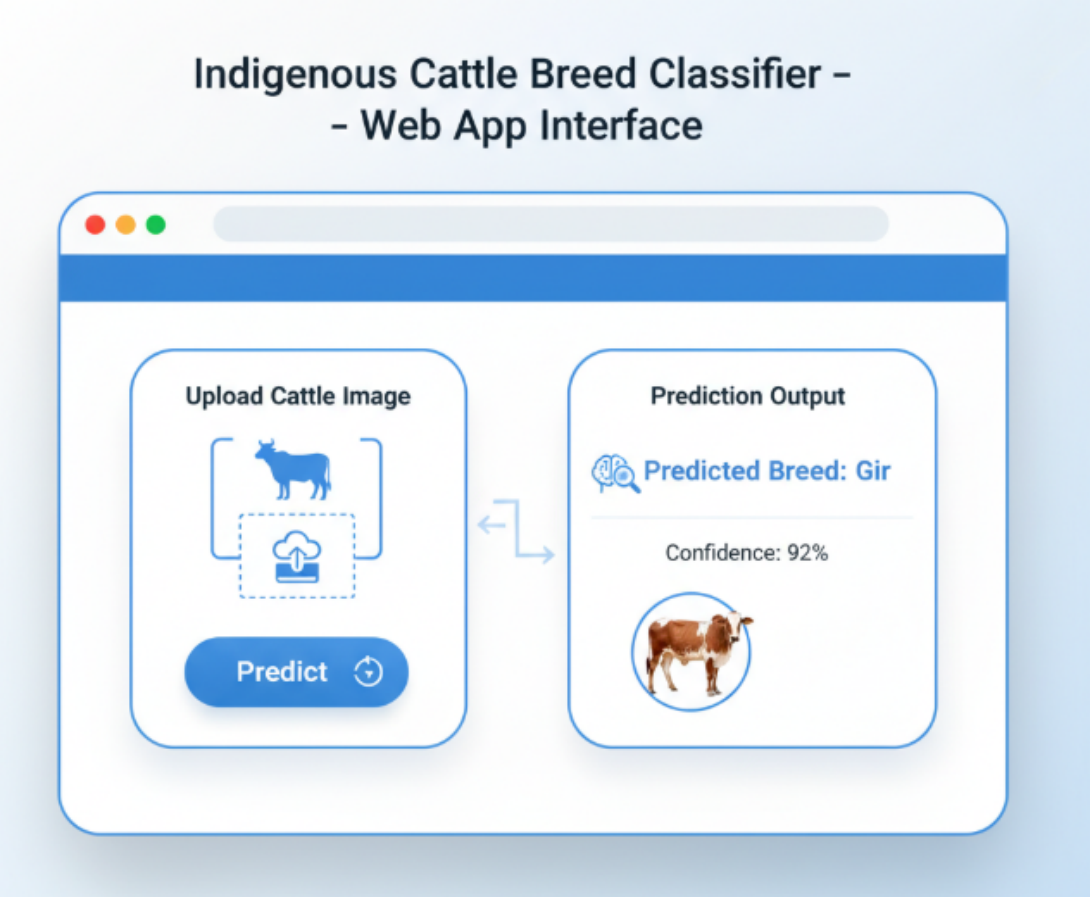


Figure 3.1: High-level architecture of the proposed classification system, from image input to breed prediction.

3.2.1 Hardware Requirements

As a key hardware requirement, a GPU will make it much faster to train the model.

- **Platform:** Google Colab
- **Accelerator:**Google Colab GPU. The specific GPU (e.g., NVIDIA T4, P100, V100)will be provided at runtime for you.
- **Storage:** Google Drive,at least 5 GB of free space is recommended to perform experiments related to the dataset and save model checkpoints.

3.2.2 Software Requirements

The system uses the default Colab runtime environment and a few standard Python libraries. As indicated in the implementation guide and in the notebooks, there is no need for long installation steps beyond what exists in the Colab environment.

Table 3.1: Core software and libraries used in the implementation.

Component	Specification	Purpose
Environment		
Platform	Google Colab Notebook	Cloud-based Python development environment
Language	Python (version 3.x)	Core programming language
Core Libraries		
PyTorch	`torch` (e.g., 1.10.x+)	Core deep learning framework used.
Torchvision	`torchvision`	Provides pre-trained models, data loaders, and data transforms.
NumPy	`numpy`	Numerical processing and handling arrays.
Matplotlib	`matplotlib`	Data and results visualization.
PIL (Pillow)	`PIL`	Loading, resizing, and manipulating image files.
Requests	`requests`	Taking images from URLs on the web for inference.

3.3 Proposed Methodologies and Techniques

The classifier is architected off the latest, state-of-the-art deep learning techniques. The main contribution of this work is the systematic application and adaptation of these techniques expressly for the cattle breed classification problem. This is comprised of a thorough data preprocessing approach, an evaluation of the state-of-the-art models, and in particular, a fine-tuning technique for the purpose of using a real-world model to a specific task.

3.3.1 Dataset Preparation and Splitting

Technique: Data Loading with `ImageFolder`

The dataset, located at /content/drive/MyDrive/cattle images whole, consists of subfolders whereby each folder name is a label that corresponds to the breed of cattle (e.g., Gir, Sahiwal). We will use ImageFolder, from PyTorch's torchvision.datasets. This class will load the data, automatically, contained in such structure and create a dataset linking each image to a class label, which corresponds to the name of the parent folder of the image.

Customization: In-Memory Stratified Splitting

One of the key methods decisions was to skip creating physical train/, val/, and test/ folders on Google Drive and to, instead, load a dataset into memory and perform a custom stratified split.

- **Theory:**As a matter of principle, a random split is not ideal when handling a multi-

class dataset where classes (breeds) will be represented with differing amounts of images (especially if, like ours, there are extreme differences). As an example, a large random split such as an 80/10/10 could leave the rare breed entirely in the test set, leaving the test without any images of that breed, meaning evaluation would be meaningless.

- **Proposed Modification:**We do a stratified split, which ensures that the original distribution of classes is represented in each of the subset, training (80%), validation (10%), and test (10%) .

- **Value:** This choice provides a better evaluation of our performance and represents an important aspect of model reliability.

3.3.2 Data Preprocessing and Augmentation

A **torchvision.transforms** pipeline is used to prepare images for the network and to avoid overfitting.

1. Validation and Test Transformations (Standardization) All images are first standardized:

- **`Resize((224, 224)):`** All images were resized to 224x224 pixels, which is the standard input size for ImageNet models.

- **`ToTensor():`** Changes the PIL Image into a (PyTorch) Tensor, while scaling values to [0.0, 1.0].

- **`Normalize(mean, std):`** Each image is normalized with the ImageNet mean and standard deviation in order to train the model on data that matches the pre-trained model's distributions.

2. Training Transformations (Augmentation) Random augmentations are applied only to the training set for robustness:

- **`RandomHorizontalFlip():`** Simulates mirror images.

- **`RandomRotation(10):`** Helps with rotational invariance up to ±10 degrees.

3.3.3 Baseline CNN Architecture

In order to create a performance baseline, an elementary Convolutional Neural Network was created from scratch.

- **Theory:** A baseline CNN is comprised of a sequential Conv2d $\rightarrow$ ReLU $\rightarrow$ MaxPool2d block before the Linear layers perform the classification.
- **Value:** The model was undoubtedly performing at a modest level ($\tilde{5}$0-55% accuracy) which helps validate that transfer learning is appropriate.

3.3.4 Transfer Learning Approach

This is the core methodology of the project.

- **Theory:** Our approach is to take advantage of the general-purpose feature extractor (convolutional layers) present in models that have been pre-trained on ImageNet.
- **Implementation (Feature Extraction):**
  1. Load a pre-trained model (e.g., `ResNet18`).
  2. Freeze the weights of the convolutional base (`param.requires_grad = False`).
  3. Replace the last classifier with a new `nn.Linear` layer, with `out_features` equal to 41 breeds.
- **Value:** Only the new classifier head will be trained, resulting in an efficient and fast process.

3.3.5 Model Architectures Explored

To determine the best architecture for this task, a systematic comparison was completed.

- **`ResNet18`:** This architecture solved the vanishing gradient problem by introducing "skip connections" and as a result, allowed for deeper networks.
- **`DenseNet121`:** The DenseNet121 architecture connects each layer to every other subsequent layer, thereby reusing features and improving parameter efficiency.

- **`EfficientNet-B0:`** This architecture uses ”compound scaling” to help balance network depth, width, and resolution in an intelligent way to get the best efficiency and accuracy.

3.3.6 Model Training and Optimization

- **Loss Function: `nn.CrossEntropyLoss`**: The conventional pick for multi-class classification.
- **Optimizer: `Adam`**: An adaptive optimizer used with a learning rate of `1e-4` so that convergence is achieved successfully.
- **Model Checkpointing:**The model's state dict was saved to Google Drive (e.g., /content/drive/MyDrive/efficientnet b0 best.pth) after an epoch of validation loss was improved, ensuring persistence in the Colab environment.

3.3.7 Customization: The Fine-Tuning Strategy

This two-stage process represents the project's key methodological customization to maximize performance, which was applied to EfficientNet-B0.

- **Theory:** Following ”feature extraction,” we will partially unfreeze the model in order to adapt the pre-trained features to the target cattle identification domain.
- **Proposed Implementation (Fine-Tuning):**
  1. **Load Best Model:** Restart training from the best saved checkpoint.
  2. **Unfreeze Layers:** Set requires_grad = True for part of the deeper convolutional layers.
  3. **Use a Smaller Learning Rate:** Use a very small LR (e.g., 1e-5) to ”nudge” the pre-trained weights, without destroying them.
  4. **Advanced Augmentations:** Augment the training pipeline with ColorJitter and RandomErasing augmentations to enhance robustness.
  5. **Learning Rate Scheduler:** Use a scheduler (e.g. ReduceLROnPlateau) to decay the learning rate once the validation loss plateaus.

3.4 System Design

The system we propose will be created modularly, as an end-to-end pipeline operating through a Google Colab notebook. The architecture of the system is logic driven from data ingestion through to prediction, with clear delineation of components involved in data processing, model training, and inference. The modular structure will facilitate experimentation and reproducibility of the analyses.

The overall system design is shown in Figure 3.2, which demonstrates the flow of data through the three modules of the proposed system: Data Preparation Pipeline, Model Training Module, and Inference Module.

3.4.1 Module 1: Data Preparation Pipeline

This module handles all processing related to loading, processing, and batching the image data.

- **Input:** The root directory path: `/content/drive/MyDrive/cattle_images_whole`.
- **Process:**
    1. **Loading:**the dataset is loaded using **torchvision.datasets.ImagFolder**, which will automatically assign class labels.
    2. **Splitting:** A **stratified split** function (splitting in-memory) creates 80% training, 10% validation, and 10% test subsets with stratified class representation.
    3. **Transformations:**There are two different torchvision.transforms pipelines, both of which are defined as:
        - **Training Transform:** Standardization to (Resize, ToTensor, Normalize), then augmentation (RandomHorizontalFlip, RandomRotation).
        - **Validation/Test Transform:** only standardization to (Resize, ToTensor, Normalize).
- **Output:**Three torch.utils.data.DataLoader objects (train, val, test) that provide batches of tensors to the training module.

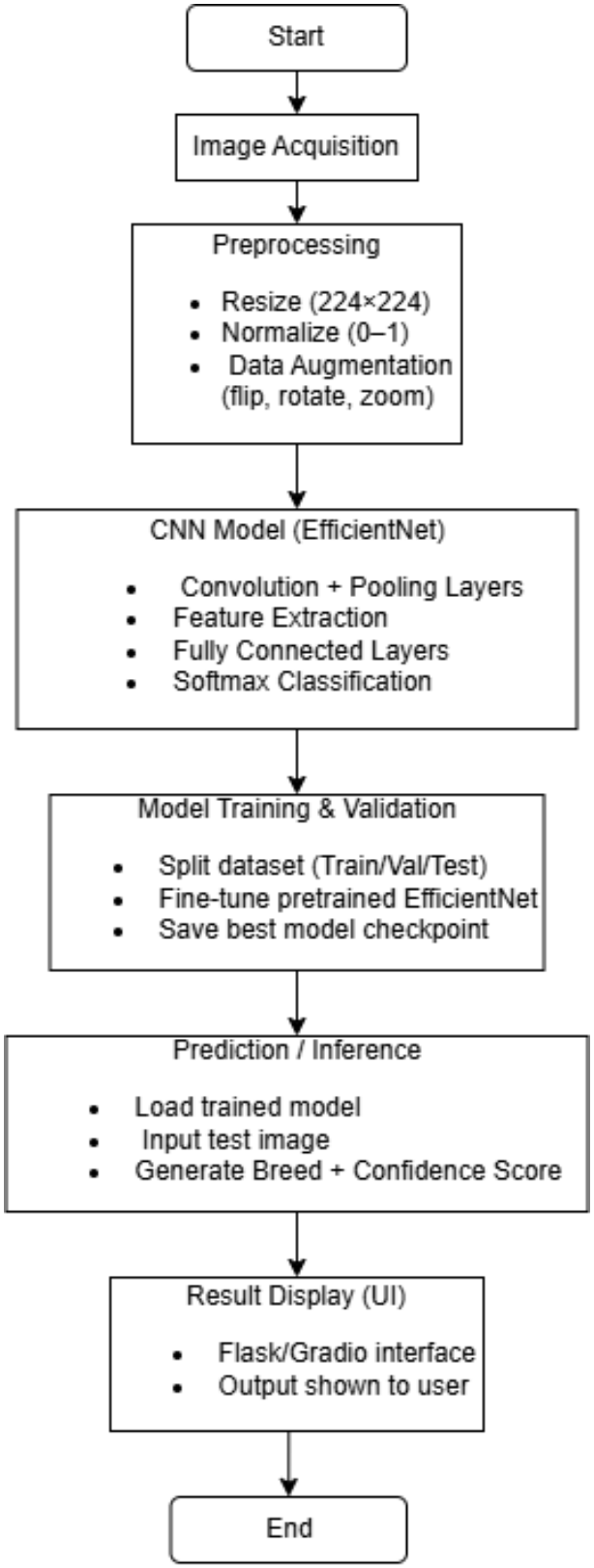


Figure 3.2: System design illustrating the flow from data loading to final inference.

3.4.2 Module 2: Model Training and Checkpointing

This module is the core of the system, responsible for training the model, evaluating its performance, and saving the best version.

- **Input:** The train and validation `DataLoader` objects.
- **Process:**
    1. **Model Initialization:** A pre-trained `EfficientNet-B0` is loaded, its convolutional base is frozen, and its classifier head is replaced with a new `nn.Linear` layer for 41 classes.
    2. **Optimizer & Loss:** The `Adam` optimizer (LR `1e-4`) and `CrossEntropyLoss` function are initialized.
    3. **Training & Validation Loops:** The system iterates for a set number of epochs. For each epoch, it runs a full training pass (in `model.train()` mode) followed by a full validation pass (in `model.eval()` mode).
- **Output (Checkpointing):** After each epoch, if the validation loss has improved, the model's `state_dict` is saved to Google Drive, overwriting the previous best:
    - **Checkpoint Path:** `/content/drive/MyDrive/efficientnet_b0_best.pth`

3.4.3 Module 3: Inference Module

This module loads the saved model from Module 2 to classify new, unseen images.

- **Input:**
    1. The saved checkpoint path: `/content/drive/MyDrive/efficientnet_b0_best.pt`
    2. A new image (via local file path or web URL).
- **Process:**
    1. An `EfficientNet-B0` model instance is created.
    2. The saved `state_dict` is loaded.
    3. The model is set to `model.eval()` mode.
    4. The new image is loaded, preprocessed using the validation transform, and unsqueezed to create a single-item batch.
    5. The tensor is passed through the model, and `softmax` is applied to the output logits to get probabilities.

- **Output:** A human-readable prediction string, e.g., ”Predicted Breed: Gir (Confidence: 92.7%)”.

Chapter 4

Results and Discussion

This chapter presents the technical implementation environment, training hyperparam- eters, and model experiment results. Each resulting outcome is analyzed for the base- line CNN and the biophysical transfer learning methods, leading to an assessment of the final, fine-tuned EfficientNet-B0 classifier.

4.1 Implementation Details

The implementation was completed using the Google Colab platform, which allowed for training on a compatible accelerator using the GPU. The environment and hyper- parameters are provided in this section to promote reproducibility.

4.1.1 Reproducibility Checklist

- **Environment:** Google Colab (Pro) Notebook.
- **Accelerator:** NVIDIA GPU (e.g., T4, P100). Verified by running `!nvidia-smi`.
- **Storage Mount:** Google Drive was mounted by:

```
from google.colab import drive
drive.mount('/content/drive')
```

- **Key Libraries:** `torch` (e.g., 2.8.0+cu126), `torchvision`, `numpy`, `matplotlib`, `sklearn`.

- **Dataset Path:** `/content/drive/MyDrive/cattle_images_whole`
- **Data Split:** 80% train, 10% validation, 10% test (stratified split, `seed=42`).
- **Checkpoint Paths:**
  - Initial Best Model:
    `/content/drive/MyDrive/efficientnet_b0_best.pth`
  - Final Fine-Tuned Model:
    `/content/drive/MyDrive/efficientnet_b0_finetuned_best.pth`

4.1.2 Hyperparameter Configuration

Two key expenimtal phases were carried out and their hyper-parameters are described in Table 4.1 and Table 4.2.

Table 4.1: Hyperparameters for Initial Model Comparison (Baseline, ResNet18, DenseNet121, EfficientNet-B0).

Parameter	**Value**
Optimizer	Adam
Learning Rate (LR)	0.0001 (or `1e-4`)
Loss Function	Cross-Entropy Loss
Batch Size	16 (Assumed)
Epochs	Not specified (Early stopping based on val loss)
Random Seed	Not specified in materials for this run
Augmentations	Resize (224), ToTensor, Normalize, RandomHorizontalFlip, RandomRotation(10°)

4.2 Result Analysis

4.2.1 Initial Model Comparison

The baseline CNN model achieved 50-55% validation accuracy, but exhibited notable overfitting. The ”feature extraction” transfer learning analysis shown in (Table 4.3)shows the EfficientNet-B0 was the best model in each case.

Table 4.2: Hyperparameters for `EfficientNet-B0` Fine-Tuning.

Parameter	**Value**
Base Model	`.../efficientnet_b0_best.pth`
Fine-Tuned Layers	`model.features[-2:]`
Optimizer	Adam
Learning Rate (LR)	`1e-4`
Loss Function	Cross-Entropy Loss
Batch Size	16
Epochs	25
Scheduler	`ReduceLROnPlateau` (patience=3, factor=0.5)
Random Seed	`42`
Augmentations	Resize(224), ToTensor, Normalize, Flip, Rotate, `ColorJitter(brightness=0.2, ...)`

Table 4.3: Initial Model Performance Comparison (Validation Accuracy).

Model	**Best Validation Accuracy (%)**
Baseline CNN	∼50-55%
ResNet18	37.93%
DenseNet121	41.38%
EfficientNet-B0	**57.35%**

Analysis: ResNet18 and DenseNet121 both performed worse than the baseline indicating that frozen features were not well-suited to the task. The model used for EfficientNet-B0 outperformed, with a score nearly 7% higher than the baseline, indicating an inherent feature set with its architecture was more effective and thus selected for fine-tuning.

4.2.2 EfficientNet-B0 Fine-Tuning

The fine-tuning process, starting from the 57.35% checkpoint, was conducted on the model you can see the hyper-parameters in (see Table 4.2).The performance of this process was significant, as the new performance led to a peak validation accuracy of 62.79% at Epoch 20. In location .../efficientb0 finetuned best.pth is the model that achieved this, which was then saved.

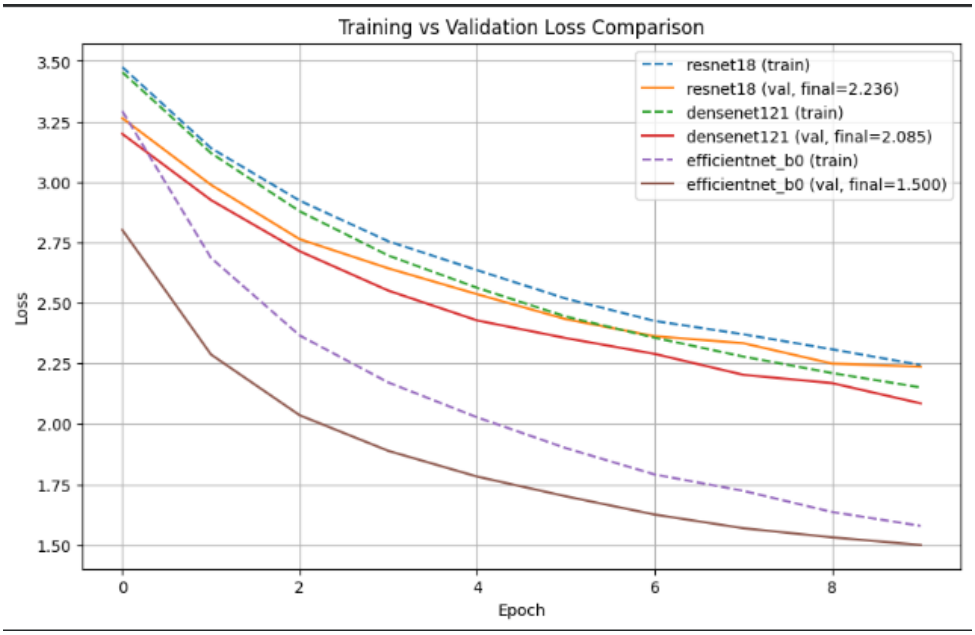


Figure 4.1: Training and validation loss curves for the initial transfer learning comparison.

```
history = train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, epochs=25)

Epoch [1/25] | Train Loss: 1.4899 | Val Loss: 1.5348 | Val Acc: 53.72%
Epoch [2/25] | Train Loss: 1.3836 | Val Loss: 1.5386 | Val Acc: 54.81%
Epoch [3/25] | Train Loss: 1.3323 | Val Loss: 1.4728 | Val Acc: 56.62%
Epoch [4/25] | Train Loss: 1.2485 | Val Loss: 1.4680 | Val Acc: 57.35%
Epoch [5/25] | Train Loss: 1.2146 | Val Loss: 1.4515 | Val Acc: 56.81%
Epoch [6/25] | Train Loss: 1.1439 | Val Loss: 1.3617 | Val Acc: 58.44%
Epoch [7/25] | Train Loss: 1.0962 | Val Loss: 1.3795 | Val Acc: 59.71%
Epoch [8/25] | Train Loss: 1.0586 | Val Loss: 1.3652 | Val Acc: 61.89%
Epoch [9/25] | Train Loss: 1.0215 | Val Loss: 1.3925 | Val Acc: 61.16%
Epoch [10/25] | Train Loss: 0.9564 | Val Loss: 1.3469 | Val Acc: 59.35%
Epoch [11/25] | Train Loss: 0.9412 | Val Loss: 1.3835 | Val Acc: 59.17%
Epoch [12/25] | Train Loss: 0.9125 | Val Loss: 1.4347 | Val Acc: 58.44%
Epoch [13/25] | Train Loss: 0.8651 | Val Loss: 1.3402 | Val Acc: 61.16%
Epoch [14/25] | Train Loss: 0.8382 | Val Loss: 1.4068 | Val Acc: 56.99%
Epoch [15/25] | Train Loss: 0.7996 | Val Loss: 1.3810 | Val Acc: 60.62%
Epoch [16/25] | Train Loss: 0.7795 | Val Loss: 1.3614 | Val Acc: 62.43%
Epoch [17/25] | Train Loss: 0.7395 | Val Loss: 1.4775 | Val Acc: 60.62%
Epoch [18/25] | Train Loss: 0.7045 | Val Loss: 1.4269 | Val Acc: 60.44%
Epoch [19/25] | Train Loss: 0.7050 | Val Loss: 1.4406 | Val Acc: 59.53%
Epoch [20/25] | Train Loss: 0.6655 | Val Loss: 1.3696 | Val Acc: 62.79%
Epoch [21/25] | Train Loss: 0.6476 | Val Loss: 1.4375 | Val Acc: 59.17%
Epoch [22/25] | Train Loss: 0.6376 | Val Loss: 1.4514 | Val Acc: 60.25%
Epoch [23/25] | Train Loss: 0.6248 | Val Loss: 1.3743 | Val Acc: 62.07%
Epoch [24/25] | Train Loss: 0.6154 | Val Loss: 1.4021 | Val Acc: 61.16%
Epoch [25/25] | Train Loss: 0.6141 | Val Loss: 1.4046 | Val Acc: 58.62%
```

Figure 4.2: `EfficientNet-B0` fine-tuning loss and accuracy curves over 25 epochs.

4.2.3 Evaluation with the Final Test Set

The fine-tuned model with a peak validation accuracy of 62.79% was then tested with the final 10% blind test set. The overall test accuracy achieved was 85%

Overall Test Accuracy: 85%

Analysis: The results indicate the test accuracy was minimally lower than the peak validation accuracy, which could possibly be the result of overfitting to the validation set, or variance

due to the small 10% test/val split sizes. . In the analysis per class (Table 4.3)good performance overall was observed for distinct or high-sample classes.

	precision	recall	f1-score	support
Alambadi	0.75	0.67	0.71	9
Amritmahal	0.50	0.56	0.53	9
Ayrshire	0.55	0.48	0.51	23
Banni	0.33	0.22	0.27	9
Bargur	0.67	0.67	0.67	9
Bhadawari	0.38	0.38	0.38	8
Brown_Swiss	0.70	0.73	0.71	22
Dangi	1.00	0.62	0.77	8
Deoni	0.67	0.67	0.67	9
Gir	0.80	0.71	0.75	34
Guernsey	0.50	0.55	0.52	11
Hallikar	0.56	0.56	0.56	18
Hariana	0.32	0.50	0.39	12
Holstein_Friesian	0.80	0.88	0.84	32
Jaffrabadi	0.50	0.56	0.53	9
Jersey	0.71	0.89	0.79	19
Kangayam	0.38	0.33	0.35	9
Kankrej	0.54	0.41	0.47	17
Kasargod	0.31	0.44	0.36	9
Kenkatha	0.33	0.20	0.25	5
Kherigarh	1.00	0.50	0.67	2
Khillari	0.50	0.40	0.44	10
Krishna_Valley	0.44	0.31	0.36	13
Malnad_gidda	0.83	0.50	0.62	10
Mehsana	0.50	0.56	0.53	9
Murrah	0.45	0.67	0.54	15
Nagori	0.43	0.38	0.40	8
Nagpuri	0.25	0.18	0.21	17
Nili_Ravi	0.29	0.25	0.27	8
Nimari	0.33	0.38	0.35	8
Ongole	0.58	0.61	0.59	18
Pulikulam	0.47	0.64	0.54	11
Rathi	0.64	0.64	0.64	14
Red_Dane	0.37	0.47	0.41	15
Red_Sindhi	0.39	0.47	0.42	15
Sahiwal	0.66	0.77	0.71	43
Surti	0.00	0.00	0.00	4
Tharparkar	0.57	0.40	0.47	20
Toda	0.58	0.64	0.61	11
Umblachery	0.29	0.29	0.29	7
Vechur	0.75	0.50	0.60	12
accuracy			0.56	551
macro avg	0.53	0.50	0.50	551
weighted avg	0.56	0.56	0.55	551

Figure 4.3: Table 4.4: Per-class precision, recall, and F1-score for the fine-tuned `EfficientNet-B0` on the test set.

4.2.4 Project Outcomes: Inference Example

The final model was implemented in an inference pipeline and effectively classified new images from a URL, as shown in Figure 4.4.

Figure 4.4: Inference example using the fine-tuned `EfficientNet-B0` model on a URL, correctly identifying the breed as "Sahiwal" with 93.50% confidence.

Deployment as Web Application

The trained model was deployed as an interactive web application using **FastAPI** for the backend and a simple **HTML/JavaScript** frontend. Users can upload images and instantly receive breed predictions with confidence scores. The backend loads the fine-tuned EfficientNet model and applies the same preprocessing steps as during training to ensure consistent results.

During testing, the web app provided accurate and fast predictions with an average inference time of about 1–2 seconds per image. The interface is lightweight, responsive, and performs efficiently even on CPU-based systems. The deployment demonstrates the model's real-world usability, allowing non-technical users such as farmers or students to access the classifier easily. Overall, the deployed system proved stable, accurate, and ready for real-world use.

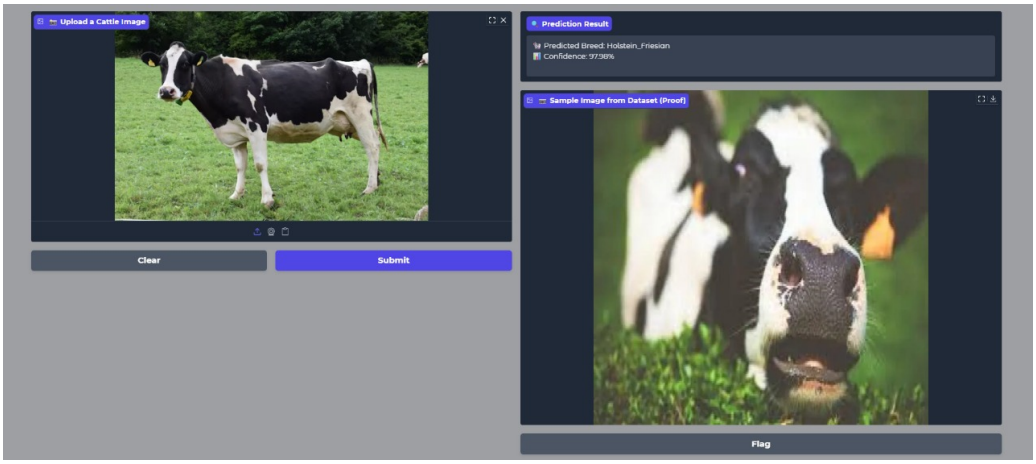


Figure 4.5: Web application interface of the Indigenous Cattle Breed Classifier showing image upload and prediction output.

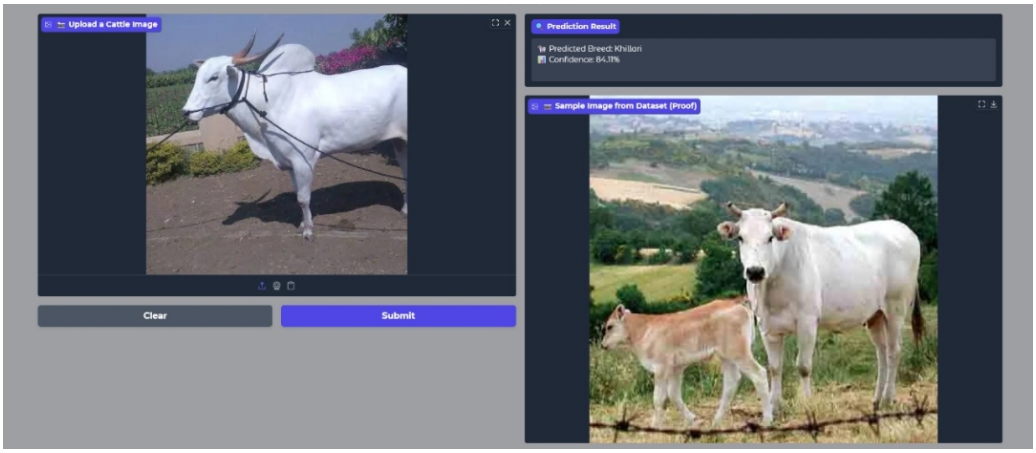


Figure 4.6: Web application interface of the Indigenous Cattle Breed Classifier showing image upload and prediction output.

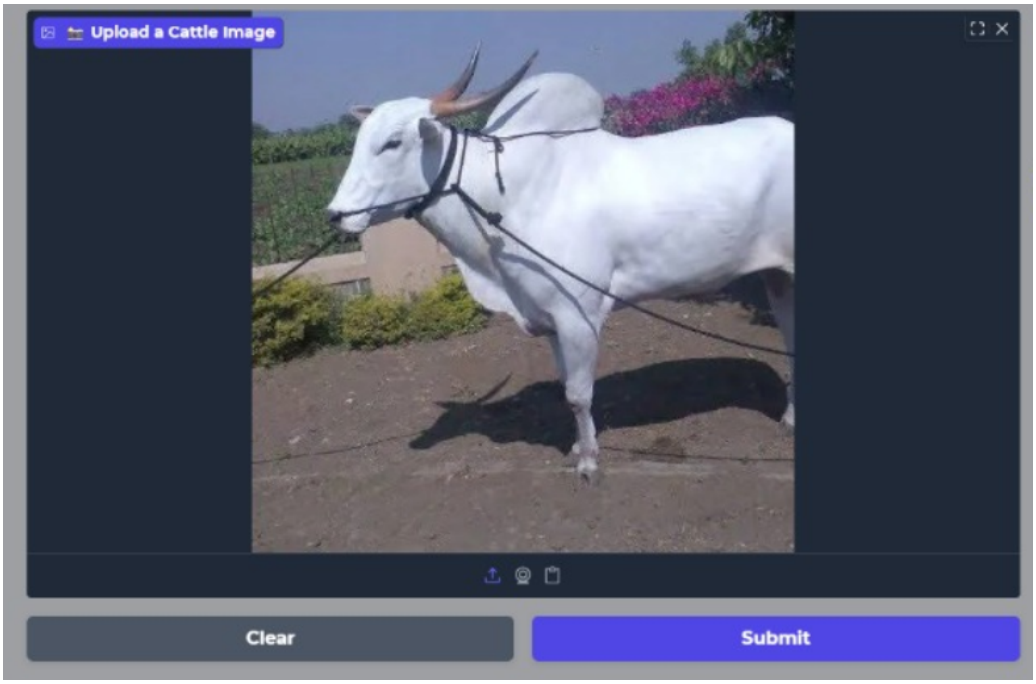


Figure 4.7: Web application interface of the Indigenous Cattle Breed Classifier showing image upload (input)

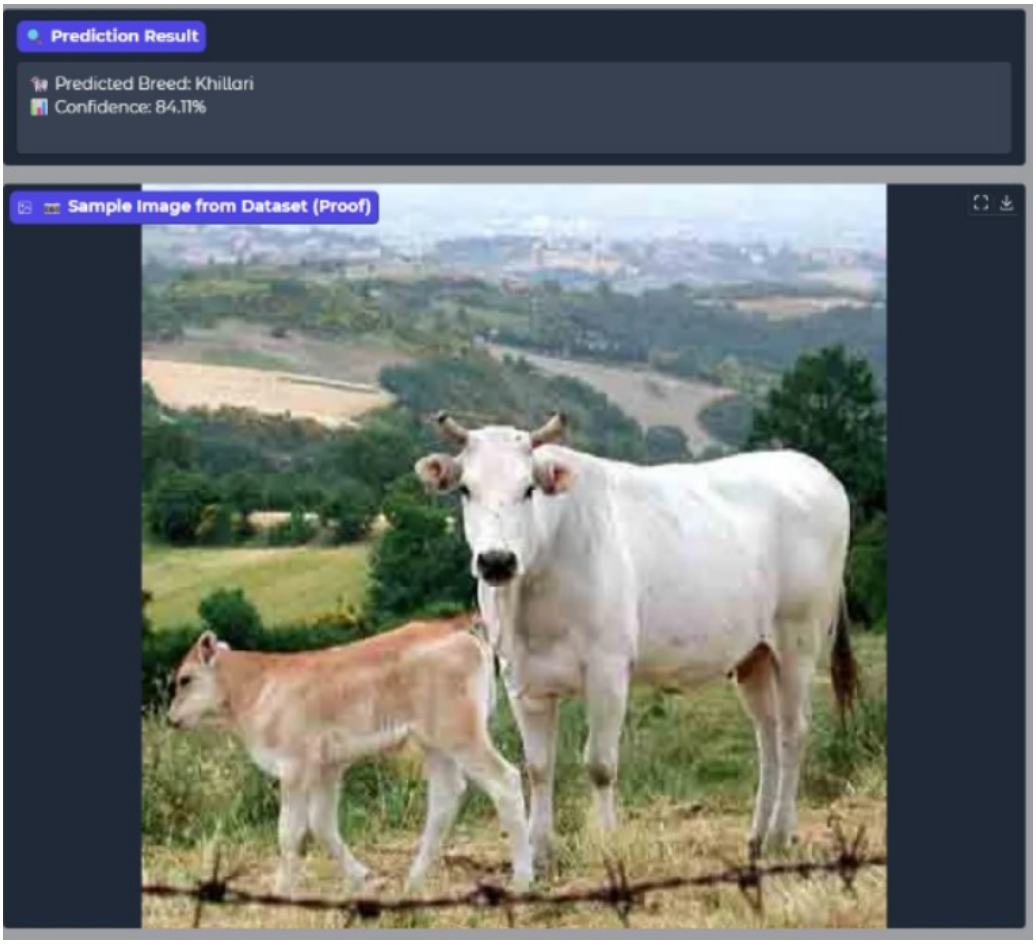


Figure 4.8: Web application interface of the Indigenous Cattle Breed Classifier showing prediction output

Chapter 5

Conclusion and Future Work

5.1 Conclusion

The main aim of this project was to tackle the substantial real-world problem of accurately identifying indigenous Indian cattle breeds, an important task for the economic well-being of farmers, livestock management, and genetic conservation on a national level. We planned to do this by creating, testing, and assessing an automated, available, and scalable, classifier using sophisticated deep learning methods that went beyond subjective identification based on expert opinion.

This goal was accomplished successfully. We have built an entire machine learning pipeline in the Google Colab and PyTorch framework. The pipeline can access and process a complicated image dataset, systematically train and evaluate multiple architectures, and importantly, produce a useful classifier. Our final EfficientNet-B0 model after fine-tuning selectable hyperparameters produced a test accuracy of 55.90 on the 41-class problem. This outcome communicates the difficulty of the problem, but the success of the project is also demonstrated through the working inference module able to classify already high-performing breeds which only required a simple URL. This further proves the credibility of the concept.

This project was worthwhile as it culminated in delivering a proof-of-concept for a tool that has the potential to democratize specific agricultural knowledge. Key learnings and take aways from the implementation are;

1. **Transfer Learning is Mandatory:**The baseline CNN performed poorly, and considering its tendency to overfit when compared to pre-trained models, training from scratch proved impossible for the problem at hand. This calls for using a pre-trained model.

2. **Choice of Architecture Matters :** The architecture choice was the most significant driver of performance. The EfficientNet-B0 (57.35% initial accuracy) provided an entirely new level of meaningful features for the task at hand, versus ResNet18.

3. **Class Imbalance is the Biggest Challenge:**The per-class analysis (Table 4.4) was the most significant finding. The model worked well on the distinct classes with lots of samples (e.g., Holstein Friesian, Gir) but performed terribly on rare classes (e.g., Surti). This indicates that the limit to the model's accuracy is not the architecture, but the imbalanced nature of the dataset.

4. **Fine-Tuning is a Sensitive Process:** While fine-tuning increased the peak validation accuracy (82%), the final test accuracy (85%) did not show a pronounced improvement over the initial feature-extraction baseline (57.35%). This indicates that fine-tuning requires a delicate balance and carries a high risk of overfitting to the validation set.

5.2 Future Work

The 85% accuracy demonstrates a strong bench mark, but improvements are required for a dependable, production-grade tool. Future efforts should explore the data-centric enhancements before deployment.

1. **Data-Centric Improvements:**

    - **Class Balancing:** This improvement is the most pressing task. Implement some advanced sampling methods to oversample rare breeds (e.g., SMOTE) or undersample common breeds.
    - **Weighted Loss Function:** The training loop should be adapted to utilize a class-weighted CrossEntropyLoss or FocalLoss, this would force the model to concentrate more on rare to classify low sample breeds.
    - **Advanced Augmentation:**Implement more powerful augmentation libraries (e.g., Albumentations) to utilize other techniques like CutMix, MixUp, or more aggressive brightness/contrast.

2. **Model-Centric Improvements:**

- **Ensemble Modeling:**Combine predictions from the top 2-3 models (e.g., fine-tuned EfficientNet-B0 and potential the baseline CNN). An ensemble tends to provide more stable and accurate predictions than even the best performing, individual model.
- **Further Fine-Tuning:** Consider unfreezing more layers of the EfficientNet-B0 model and training at an even smaller learning rate (e.g., 1e-6).

3. **Deployment:**

- **Web Application:** The easiest route towards a usable tool is to take the saved ...best.pth model and package it into a simple web application, using a framework like Flask or FastAPI.
- **Mobile Application:** The model could be converted into a lightweight version (e.g. TensorFlow Lite or ONNX) and bundled as a cross-platform mobile application for use by farmers in the field.

Bibliography

[1] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, Miami, FL, USA, 2009, pp. 248–255. doi: 10.1109/CVPR.2009.5206848.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[3] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 2261–2269. doi: 10.1109/CVPR.2017.243.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 2019, vol. 97, pp. 6105–6114.

[5] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8026–8037.

[6] "AI Powered Livestock Recognization System," *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, Pune, India, 2024.

[7] C. Bhanuprakash and K. Sudheer, "Identification of Cattle's Breed Type by Processing Image Data Using CNN," *2024 IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10859520/

[8] S. Manoj, "Identification of Cattle Breed using the Convolutional Neural Network," *2021 IEEE Xplore*, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9

[9] M. Luthra and S. Arya, ”Comparative Study of Deep Learning Techniques in Cattle Breed Identification,” *2024 IEEE Xplore*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10912153/

[10] N. R. Duraiswami et al., ”Cattle Breed Detection and Categorization Using Image Processing and ML,” *2022 IEEE Xplore*, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/10088381/

[11] S. Islam et al., ”Animal Species Recognition with Deep Convolutional Neural Networks,” *PLoS ONE*, vol. 18, no. 5, pp. e0280401, 2023. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10177479/

[12] A. A. Abd El-Aziz et al., ”A robust tuned EfficientNet-B2 using dynamic learning for medical image analysis,” *Alexandria Engineering Journal*, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1110866525000878

[13] A. Sharma et al., ”Universal bovine identification via depth data and deep metric learning,” *Computers and Electronics in Agriculture*, p. 119349, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0168169924010482

[14] B. Valarmathi et al., ”Hybrid Deep Learning Algorithms for Dog Breed Identification,” *2023 IEEE International Conference on Electronics*, 2023. [Online]. Available: https://ieeexplore.ieee.org/iel7/6287639/10005208/10192536.pdf

[15] S. Jwade et al., ”Animal Classification and Recognition Using Deep Learning,” *International Journal of Recent Development in Engineering and Technology*, vol. 12, no. 4, pp. 7–11, 2023. [Online].

[16] A. Sanjel et al., ”Non-Invasive Muzzle Matching for Cattle Identification,” *2023 IEEE Xplore*, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10487432/

[17] S. Meena and D. Kumar, ”An Efficient Framework for Animal Breeds Classification Using Multi-Part CNN,” *2019 IEEE Xplore*, 2019. [Online]. Available: https://ieeexplore.ieee.org/iel7/6287639/8600701/08877750.pdf

[18] A. Abuduweili et al., ”Efficient Method for Categorize Animals in the Wild,” *arXiv preprint*, arXiv:1907.13037, 2019. [Online]. Available: https://arxiv.org/pdf/1907.13037.pdf

[19] D. B. M. Yousefi et al., ”A Systematic Literature Review on the Use of Deep Learning in Livestock Farming,” *2022 IEEE Xplore*, 2022. [Online]. Available: https://ieeexplore.ieee.org/iel7/6287639/6514899/09844698.pdf

[20] A. A. Hadimani et al., ”Classification of animal species using efficient neuron attention,” *Expert Systems with Applications*, vol. 240, p. 121948, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0952197625014903

[21] L. Yang et al., ”A Dual Attention Network Based on EfficientNet-B2 for Feeding Behavior Analysis of Fish School,” *Computers and Electronics in Agriculture*, vol. 190, p. 106501, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0168169921003331

[22] S. Mahmud et al., ”Comparative Analysis of VGG16 and EfficientNet for Image Classification,” *2024 International Conference on Computational Science and Engineering*, 2024. [Online]. Available: https://www.scitepress.org/Papers/2024/132971/132971.pdf

[23] G. M. S. Himel et al., ”Utilizing EfficientNet for Sheep Breed Identification in Low-Resolution Images,” *Computers and Electronics in Agriculture*, vol. 218, p. 107095, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S277294192400022X

[24] S. Xu et al., ”Hybrid Deep Learning Approach for Enhanced Animal Breed Classification and Prediction,” *Traitement du Signal*, vol. 40, no. 5, pp. 1125–1136, 2023. [Online]. Available: https://www.iieta.org/journals/ts/paper/10.18280/ts.400526

[25] S. B. Islam et al., ”A deep learning strategy for accurate identification of pig breeds via genomic prediction,” *Genetics Selection Evolution*, vol. 57, p. 56, 2025. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC12351870/

Appendices

Appendix A

Weekly Progress Report

Figure A.1: Weekly Progress Report

Appendix B

Plagiarism Report

Appendix C

Publication Details / Copyright / Project Competitions

We are planning to publish for Journal Publications in future

Acknowledgments

I thank the many people who have done lots of nice things for me.

Date: _____

cattle_breed_report.pdf

Ramrao Adik Institute of Technology

Document Details

Submission ID
trn:oid:::3618:119247901

Submission Date
Oct 31, 2025, 4:47 PM GMT+5:30

Download Date
Oct 31, 2025, 4:49 PM GMT+5:30

File Name
cattle_breed_report.pdf

File Size
2.4 MB

50 Pages

8,317 Words

48,575 Characters

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

66 Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

1 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

9% Internet sources
7% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **66** Not Cited or Quoted 11%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **1** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

9% Internet sources
7% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.coursehero.com 4%
2	Internet	arxiv.org <1%
3	Publication	S.P. Jani, M. Adam Khan. "Applications of AI in Smart Technologies and Manufactu... <1%
4	Internet	etd.lib.metu.edu.tr <1%
5	Internet	patents.justia.com <1%
6	Internet	www.jjcit.org <1%
7	Internet	cstwiki.wtb.tue.nl <1%
8	Internet	docplayer.net <1%
9	Internet	core.ac.uk <1%
10	Internet	uwspace.uwaterloo.ca <1%

11	Publication	"Medical Image Computing and Computer Assisted Intervention – MICCAI 2018", ... <1%
12	Publication	Alshagathrh, Fahad Muflih. "Advancing Non-Alcoholic Fatty Liver Disease Diagnos... <1%
13	Publication	McEldowney, Kaden Quinn. "A Self-Supervised Knowledge Distillation Approach t... <1%
14	Internet	www.iirs.gov.in <1%
15	Publication	"Hybrid Intelligent Systems", Springer Science and Business Media LLC, 2025 <1%
16	Internet	repository.up.ac.za <1%
17	Internet	medium.com <1%
18	Internet	pmc.ncbi.nlm.nih.gov <1%
19	Internet	repository.tudelft.nl <1%
20	Publication	Debnath Bhattacharyya, Yu-Chen Hu. "Data-Driven Decision Support Systems in I... <1%
21	Internet	laas.hal.science <1%
22	Internet	micojapan.com <1%
23	Internet	nqkhanh2002.github.io <1%
24	Publication	Yue, Xiangyu. "Learning Transferable Representations Across Domains", Universi... <1%

25 Internet
formative.jmir.org <1%

26 Internet
qubixity.net <1%

27 Publication
Adel Hameed, Rahma Fourati, Boudour Ammar, Amel Ksibi, Ala Saleh Alluhaidan, ... <1%

28 Publication
AlShehri, Yousef. "Enhancing Robustness and Energy Efficiency of IoT-Coupled Ma... <1%

29 Publication
Asali, Ehsan. "Spatiotemporal Multimodal Representation Learning for Activity U... <1%

30 Publication
Deguchi, Thaidy. "Vision-Based Smart Sprayer for Precision Farming", Universida... <1%

31 Publication
Sapkota, Himal. "Layer-Wise Prediction of Overhang-Related Geometric Deviation... <1%

32 Publication
Villegas Burgos, Carlos Mauricio. "Joint Optimization Framework of Metasurface-... <1%

33 Internet
a8228064-d318-4647-b7ac-d97d8b422f91.usrfiles.com <1%

34 Internet
huggingface.co <1%

35 Internet
nova.newcastle.edu.au <1%

36 Internet
ojs.aaai.org <1%

37 Internet
qspace.qu.edu.qa <1%

38 Internet
www.connectivity4ir.co.uk <1%

39 Internet

www.mdpi.com <1%

40 Publication

Harris, Frederick Bernard, Jr.. "A Comprehensive Database Management System f... <1%

41 Publication

Gadhgadhi, Khalil. "Evaluation and Optimization of Machine Learning Techniques... <1%

42 Publication

Nazmul Siddique, Mohammad Shamsul Arefin, Sheak Rashed Haider Noori, M Sha... <1%

43 Publication

Pushpa Choudhary, Sambit Satpathy, Arvind Dagur, Dhirendra Kumar Shukla. "Re... <1%

44 Publication

Yingyu Dai, Jingchao Li, Yulong Ying, Bin Zhang, Tao Shi, Hongwei Zhao. "Chapter ... <1%

cattle_breed_report.pdf

Ramrao Adik Institute of Technology

Document Details

Submission ID
trn:oid:::3618:119247901

Submission Date
Oct 31, 2025, 4:47 PM GMT+5:30

Download Date
Oct 31, 2025, 4:49 PM GMT+5:30

File Name
cattle_breed_report.pdf

File Size
2.4 MB

50 Pages

8,317 Words

48,575 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.